



EduStream

Plataforma de streaming de vídeos educacionais

Garantia e Gestão da Qualidade de Software · Prof. Daniel Henrique Matos de Paiva

Pietro · João · Rafael · Gustavo · Pedro

2026 - UNA

EQUIPE

Grupo

PI

Pietro

Filas, retry, Supabase e
CI/CD

feature/processamento-
assincrono

JO

João

FFmpeg, HLS e thumbnail

feature/transcodificacao

RA

Rafael

Upload, validação e
metadados

feature/upload-videos

GU

Gustavo

Similaridade, trending e
cache

feature/recomendacoes

PE

Pedro

Player adaptativo (frontend)

feature/player-adaptativo

Evolução contínua



Recomendações

Collaborative filtering com base no histórico de visualização



Legendas automáticas

Transcrição de áudio para acessibilidade (.vtt)



Analytics

Dashboard de engajamento: tempo assistido e qualidade usada

MVP funcional e integrado — a próxima fronteira é inteligência e acessibilidade.

A barreira da qualidade do acesso educacional



Vídeos em formato único travam em conexões instáveis ou mais baixas.

Plataformas de EAD de menor porte ainda não usam streaming adaptativo (ABR)



Infraestrutura robusta é cara para instituições menores.

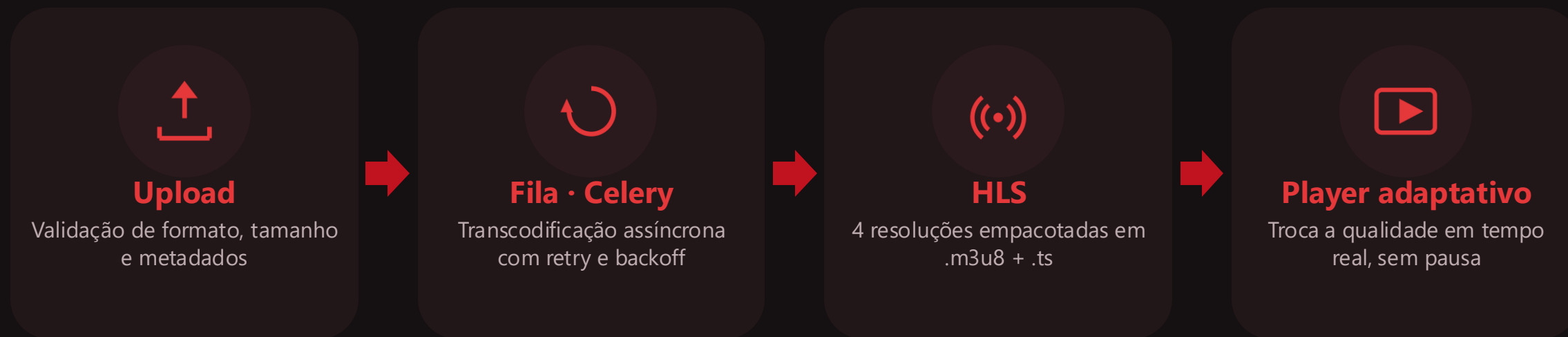
Licenciar tecnologia nível Netflix/YouTube está fora do alcance



Exclusão digital: alunos com internet limitada ficam para trás

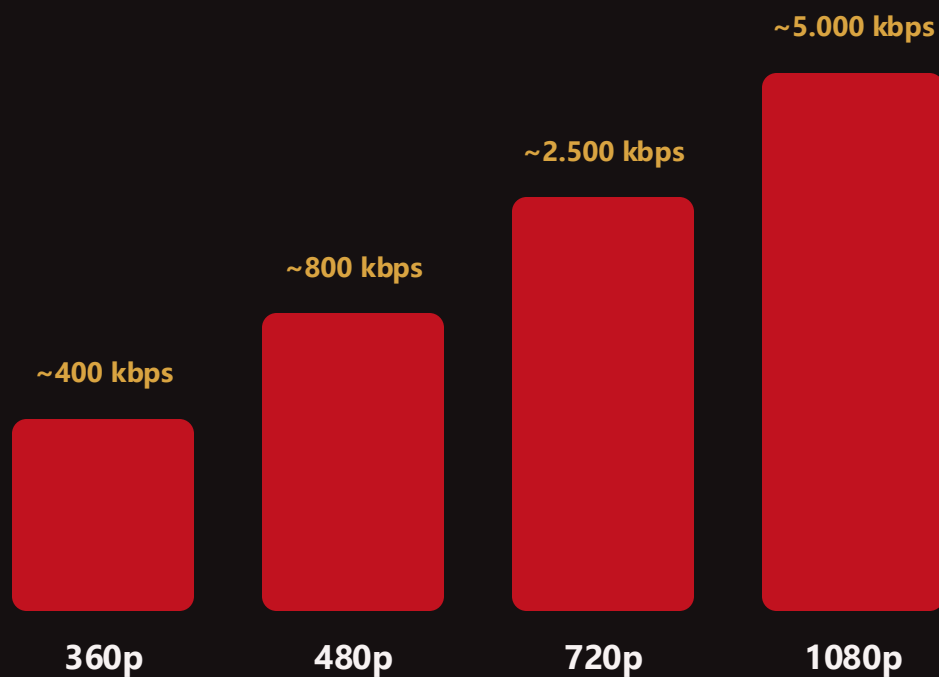
Realidade comum em regiões do Brasil com telecomunicações precárias

Streaming adaptativo e inteligente



Resposta imediata no upload: o peso roda em segundo plano e o player/layout ainda recomenda vídeos relacionados com cache em Redis.

Como o HLS adaptativo funciona



master.m3u8 • gerado na transcodificação

```
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=400000,RESOLUTION=640x360
360p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=800000,RESOLUTION=854x480
480p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=2500000,RESOLUTION=1280x720
720p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=5000000,RESOLUTION=1920x1080
1080p/playlist.m3u8
```

- O player baixa o master e escolhe a variante pela banda disponível
- Segmentos .ts curtos: trocar de qualidade não interrompe a aula
- HLS.js remede a cada download, no caso de oscilar a rede, desce um degrau

Educação de Qualidade - ODS 4

- **Assegurar educação inclusiva e equitativa**
- **Democratizar o acesso ao conhecimento em vídeo**
- **Superar barreiras de conectividade regional**



Objetivo de Desenvolvimento Sustentável 4 — ONU

"Assegurar a educação inclusiva e equitativa e promover oportunidades de aprendizagem ao longo da vida para todos."

Solução tecnológica & moderna



Backend

FastAPI + Pydantic



Filas

Redis + Celery (+ Beat)



Transcodificação

FFmpeg → HLS (.m3u8 + .ts)



DB & Storage

Supabase (Postgres +
Storage)



Frontend

HLS.js + HTML5 Video

Microsserviços desacoplados por filas: o processamento pesado nunca bloqueia a API.

`docker-compose` sobe o Redis · credenciais via `.env` (SUPABASE_URL / SUPABASE_KEY) — nunca no Git

Funcionamento/API's que juntam as peças

metadados • tabela videos (Supabase)

```
{
  "video_id": "uuid",
  "titulo": "string",
  "descricao": "string",
  "tags": ["string"],
  "categoria": "string",
  "autor": "string",
  "status": "pending | processing",
  "completed | failed",
  "criado_em": "datetime"
}
```

endpoints principais

POST	/upload	Rafael
GET	/status/{video_id}	Pietro
GET	/videos/{id}/relacionados	Gustavo
GET	/recomendacoes/{user_id}	Gustavo
GET	/trending	Gustavo
POST	/watch	Gustavo

/uploads/{video_id}/original.mp4 · /videos/{video_id}/{res}/playlist.m3u8 · master.m3u8

Desenvolvimento



GitFlow 5 branches de feature + develop + main · PRs revisados



TDD testes unitários com Pytest (e Jest no player)



CI/CD GitHub Actions: testes a cada push/PR em main e develop



Clean Code PEP 8, nomes descritivos e try/except nos pontos críticos

Pipeline real: matrix Python 3.10/3.11 · Flake8 (E9, F63, F7, F82) · pytest --cov → coverage.xml

Testes

app/upload/tests/test_upload.py

```
def test_upload_invalid_format(client):  
    r = client.post('/upload',  
                    files={'file': ('aula.txt', b'...')})  
    assert r.status_code == 400
```

```
def test_upload_max_size(client):  
    r = client.post('/upload',  
                    files={'file': ('aula.mp4', BIG)})  
    assert r.status_code == 400 # > 500 MB
```

commits semânticos + erro tratado

feat: endpoint de upload com validacao
test: testes de upload
fix: validacao de tamanho de arquivo
chore: pipeline de CI no github actions

try/except em upload, transcodificação,
filas e chamadas externas
retry: 3 tentativas, backoff 1/5/15 min

TDD de verdade: testes com estes nomes rodam no GitHub Actions a cada push/PR — matrix Python 3.10/3.11.

Obrigado!

Dúvidas?



[github.com/Pietro2820/
projeto-a3-Garantia-da-Qualidade-de-Software-2026.2](https://github.com/Pietro2820/projeto-a3-Garantia-da-Qualidade-de-Software-2026.2)